

Codeforces Round 1119 (Div. 3)

A. Moo Language School

1 second, 256 megabytes

Farmer John is trying to increase literacy rates in the United Cows of Farmer John (UCFJ). The UCFJ consists of n fields and $\frac{n}{k}$ farms (where n is a multiple of k), with each farm consisting of k consecutive fields. In other words, the i -th field is in the $\lceil \frac{i}{k} \rceil$ -th farm: Fields $1, 2, \dots, k$ are in the first farm, fields $k + 1, k + 2, \dots, 2k$ are in the second farm, etc.

Farmer John wants to build schools such that each farm has at least one school. However, some fields are owned by Farmer Nhoj, who will charge Farmer John extra to build a school there. Farmer John wants to know the minimum number of times that he would have to build a school on Farmer Nhoj's land in order to ensure that each farm has at least one school.

Input

The first line of each input contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and k ($1 \leq k \leq n \leq 20$, n is a multiple of k) — the number of fields and size of each farm.

The second line of each test case contains a binary string s of length n — the fields owned by Farmer Nhoj. If $s_i = 1$, the i -th field is owned by Farmer Nhoj. If $s_i = 0$, the i -th field is not owned by Farmer Nhoj.

Output

For each test case, output a single integer — the minimum number of times that Farmer John must build a school on Farmer Nhoj's land.

input
6 8 2 10011100 5 1 11111 8 4 01111110 5 1 00101 4 4 1101 4 4 1111
output
1 5 0 2 0 1

For the first test case, we can build a school on the 2nd, 3rd, 5th, and 7th fields, and of those, only the 5th field is owned by Farmer Nhoj, meaning our answer is 1. It can be shown that this is the best possible answer.

For the second test case, Farmer Nhoj owns every field, and since we have to build 5 schools, we must build on Farmer Nhoj's land 5 times.

B. Minus Two

2 seconds, 256 megabytes

You are given an array $a_1, a_2, \dots, a_n$. You may perform the following operation:

- For all indices i ($1 \leq i \leq n$), set $a_i = |a_i - 2|$.

Find the maximum possible frequency of any integer in a after performing the operation an arbitrary number (possibly zero) of times.

Input

The first line of each input contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of a .

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the array a .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single integer denoting the maximum possible frequency of any integer in a after performing the operation an arbitrary number of times.

input
5 2 1 3 4 1 1 1 2 3 6 7 8 4 2 2 2 2 5 1 10 100 1000 100000
output
2 3 1 4 3

In the first test case, we can perform the operation once to have $a = [|1 - 2|, |3 - 2|] = [1, 1]$, which means the maximum possible frequency of any integer in a is 2.

In the second test case, the maximum frequency of an integer in a is 3. It can be shown that, no matter how many operations we perform, the maximum possible frequency of an integer in a will never exceed 3.

C. 101

2 seconds, 256 megabytes

The score of an array b of length m is defined as the maximum length of a subarray of b such that the first and last elements of the subarray are equal to 1 and all other elements in the subarray are equal to 0. Formally, the score of b is equal to the maximum integer k for which there exists an index i such that:

- $1 \leq i \leq m - k + 1$
- $b_i = b_{i+k-1} = 1$
- $b_{i+1} = b_{i+2} = \dots = b_{i+k-2} = 0$

If there is no subarray meeting the requirements, the score of b is 0.

You are given an array $a_1, a_2, \dots, a_n$, such that each element is equal to one of $-1, 0$, or 1 . Replace each -1 with either a 0 or 1 such that the score of a is maximal over all possible ways to replace the -1 s in a .

Input

The first line of each input contains t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains n ($1 \leq n \leq 2 \cdot 10^5$) — the length of a .

The second line of each test case contains $a_1, a_2, \dots, a_n$ ($a_i \in \{-1, 0, 1\}$) — the array a .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n space separated integers representing a after the -1 s were replaced with 0s or 1s. If there are multiple possible solutions, output any.

input
10 6 1 0 -1 0 0 1 7 0 -1 0 0 1 0 1 5 -1 0 0 -1 0 4 0 0 0 0 1 -1 6 1 0 1 0 0 -1 7 0 1 0 0 0 1 0 6 -1 -1 -1 -1 -1 -1 7 -1 0 1 -1 0 0 1 3 -1 0 0
output
1 0 0 0 0 1 0 1 0 0 1 0 1 1 0 0 1 0 0 0 0 0 1 1 0 1 0 0 1 0 1 0 0 0 1 0 1 0 0 0 0 1 0 0 1 0 0 0 1 1 0 0

In the first test case, we can change the only -1 to a 0 , making $a = [1, 0, 0, 0, 0, 1]$. Since the first and last elements of a are equal to 1 , and all other elements are 0 , the score of a is 6 .

In the third test case, changing both -1 s to 1 s makes $a = [1, 0, 0, 1, 0]$, and the largest subarray that satisfies the conditions in the statement is from the 1 -st index to the 4 -th index.

In the fifth test case, we set the only -1 to 1 , making $a = [1]$, meaning the largest subarray that satisfies the conditions in the statement is the full array.

D. MEX Multiset

2 seconds, 256 megabytes

You are given an array $a_1, a_2, \dots, a_n$. There exist 3 initially empty multisets A, B, C , and for each index i ($1 \leq i \leq n$), you may put a_i into **exactly one** of A, B , or C .

Determine whether it is possible to put the elements into the multisets such that $\text{MEX}(A) + \text{MEX}(B) + \text{MEX}(C) \geq 2 \cdot \max(\text{MEX}(A), \text{MEX}(B), \text{MEX}(C))$.
*. If so, output a construction that achieves this.

* $\text{MEX}(D)$ is defined as the smallest non-negative integer that is not present in the set D . For example, $\text{MEX}([1, 2, 0, 5]) = 3$, and $\text{MEX}([1, 2, 4, 9]) = 0$. The MEX of an empty set is 0 .

Input

The first line of each input contains t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains n ($3 \leq n \leq 2 \cdot 10^5$) — the length of a .

The second line of each test case contains $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$) — the array a .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

If a valid distribution of elements into the multisets exists, output **YES**. Otherwise, output **NO**.

If the answer is **YES**, output a string s of length n on a new line, such that $s_i = \mathbf{A}$ if the i -th element was put into the multiset A , $s_i = \mathbf{B}$ if the i -th element was put into the multiset B , and $s_i = \mathbf{C}$ if the i -th element was put into the multiset C .

You can output the answer in any case (upper or lower). For example, the strings **YES**, **yes**, **yEs**, and **Yes** will be recognized as positive responses, and the strings **NO**, **no**, **No** will be recognized as negative responses. Additionally, the strings **AABCAAA**, **aabcaaa**, and **aaBcaaa** will be recognized as the same answer.

If there are multiple possible outputs, output any.

input
5 6 1 0 0 1 2 1 4 0 0 0 0 3 0 2 2 4 6 7 6 7 5 0 0 0 1 2
output
YES ABABCA YES ABAC NO YES AAAB YES ABCAB

In the first test case, we can have $A = \{0, 1, 1\}$, $B = \{0, 1\}$, $C = \{2\}$, meaning $\text{MEX}(A) + \text{MEX}(B) + \text{MEX}(C) = 4$, and $2 \cdot \max(\text{MEX}(A), \text{MEX}(B), \text{MEX}(C)) = 2 \cdot \max(2, 2, 0) = 4$.

In the third test case, it can be shown that there are no valid distributions.

E. Treasure Map Destruction (Constructive Version)

2 seconds, 256 megabytes

Bessie the Buccaneer has plundered the six or seven seas for decades, but now she needs to find a place to bury her treasure!

There are n islands in a line, and Bessie buried her treasure on some number of islands. It is guaranteed that Bessie buried treasure on at least one island.

In order to remember where she buried her treasure, she created a treasure map. The treasure map can be represented as an array of n integers, where a_i indicates the minimum number of islands that Bessie would need to travel through to get to an island with treasure if she started at island i , assuming she can only travel to adjacent islands.

However, after Bessie buried her treasure, Elsie the Evil Explorer tried to steal Bessie's treasure map! Thankfully, Elsie was unsuccessful, but in the chaos, parts of Bessie's treasure map got destroyed, meaning that some of the elements of a were set to -1 ! Panicked, Bessie gives you the destroyed array b , and wants you to find any possible set of islands that she could have placed her treasure on. It's possible that Bessie messed up while writing the treasure map, so there could be no possible set of islands that she could have placed her treasure on.

Input

The first line of the input contains t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains n ($1 \leq n \leq 2 \cdot 10^5$) — the number of islands.

The second line contains $b_1, b_2, \dots, b_n$ ($-1 \leq b_i \leq n - 1$) — the destroyed array of distances. If $b_i = -1$, the element was destroyed.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a bitstring of length n , such that the i -th character is **1** if there is a treasure at position i , or **0** if there isn't. If there are multiple possible answers, output any. If there are no possible answers, output -1 . **Your answer must contain at least one treasure.**

input
12 5 0 1 -1 -1 0 3 -1 0 2 5 -1 1 -1 1 -1 5 -1 -1 -1 -1 -1 5 -1 2 -1 3 -1 7 2 1 0 1 0 1 2 1 -1 3 1 -1 1 1 0 4 3 -1 -1 -1 6 -1 -1 0 -1 2 4 10 -1 1 -1 -1 1 1 -1 -1 2 -1
output
10101 -1 10001 11010 -1 0010100 1 010 1 0001 -1 1001001000

In the first test case, Bessie could have placed her treasure at the first, third, and fifth island, making $a = [0, 1, 0, 1, 0]$.

In the fourth test case, since the entire map was destroyed, all $2^5 - 1 = 31$ **non-empty** sets of islands are possible.

F. Binary Bubble Sort Inversions

3 seconds, 256 megabytes

Define performing a bubble on an array $b_1, b_2, \dots, b_m$ to be performing the following action:

- For each integer i ($1 \leq i < m$) **in order**, if $b_i > b_{i+1}$, swap b_i and b_{i+1} .

Define performing a reverse bubble on an array $b_1, b_2, \dots, b_m$ to be performing the following action:

- For each integer i ($1 \leq i < m$) **in reverse order**, if $b_i > b_{i+1}$, swap b_i and b_{i+1} .

For example, if we perform a bubble on $[1, 1, 0]$, we would transform it as follows: $[1, 1, 0] \rightarrow [1, 1, 0] \rightarrow [1, 0, 1]$. If we perform a reverse bubble on $[1, 1, 0]$, we would transform it as follows: $[1, 1, 0] \rightarrow [1, 0, 1] \rightarrow [0, 1, 1]$.

You are given an array $a_1, a_2, \dots, a_n$ consisting of only 0s and 1s and a binary string* s of length n . You perform n operations **in order**, where the i -th operation performs a bubble on a if $s_i = 1$ and performs a reverse bubble on a if $s_i = 0$. For each of the $n + 1$ states of a — before any operations have been performed and after each operation — count the number of inversions[†] in a .

*A binary string is a string that only contains 0s and 1s.

[†]An inversion in an array c of length m is a pair of indices i, j such that $1 \leq i < j \leq m$ and $c_i > c_j$.

Input

The first line of each input contains t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains n ($1 \leq n \leq 5 \cdot 10^5$) — the length of a and s .

The second line of each test case contains $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 1$) — the array a .

The third line of each test case contains s ($s_i \in \{0, 1\}$, $|s| = n$) — the binary string s .

It is guaranteed that the sum of n over all test cases does not exceed $5 \cdot 10^5$.

* $|c|$ denotes the length of a string c .

Output

For each test case, output $n + 1$ space separated integers, the i -th integer denoting the number of inversions in a after the first $i - 1$ operations have been performed.

input
7 4 1 1 0 0 1010 7 0 1 0 1 1 0 0 0101001 4 0 0 1 1 1001 1 1 1 5 0 1 1 0 0 11101 3 1 0 0 000 6 1 0 1 1 0 0 011000
output
4 2 1 0 0 7 4 2 0 0 0 0 0 0 0 0 0 0 0 0 4 2 0 0 0 0 2 1 0 0 7 4 2 1 0 0 0

In the first test case:

- Before any operations: $a = [1, 1, 0, 0]$, which has 4 inversions.
- After the first operation: $a = [1, 0, 0, 1]$, which has 2 inversions.
- After the second operation: $a = [0, 1, 0, 1]$, which has 1 inversion.
- After the third operation: $a = [0, 0, 1, 1]$, which has 0 inversions.
- After the fourth operation: $a = [0, 0, 1, 1]$, which has 0 inversions.

G. Index Removal

2 seconds, 256 megabytes

An array $b_1, b_2, \dots, b_m$ is good if, for each i ($1 \leq i < m$), $b_{i+1} \geq b_i$ and $b_{i+1} - b_i \leq k$. An array of length 1 is always good.

You are given an **initially good** array $a_1, a_2, \dots, a_n$. Solve the following problem for each i ($1 \leq i \leq n$) **independently**:

- The i -th element of a is removed, making $a = [a_1, a_2, \dots, a_{i-1}, a_{i+1}, \dots, a_n]$. In one operation, you are able to subtract 1 from any element of a . What is the minimum number of operations required to make a good again?

Input

The first line of each input contains t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and k ($2 \leq n \leq 2 \cdot 10^5, 1 \leq k \leq 10^9$).

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$). It is guaranteed that a is initially good.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n space separated integers on a single line: the i -th integer denoting the solution to the problem for the i -th index.

input
7 4 2 1 2 4 5 4 1 1 2 3 4 5 7 1 8 9 16 20 5 1000000000 1 6 7 67 6767 6 1 1 1 2 2 3 4 4 1 1 2 3 3 4 2 1 2 4 6
output
0 1 1 0 0 2 1 0 0 2 1 4 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 2 2 0

In the first test case:

- $i = 1, a = [2, 4, 5]$. Since a is still good, we do not need to perform any operations.
- $i = 2, a = [1, 4, 5]$. If we perform the operation once on the 2nd element, $a = [1, 3, 5]$, which is a good array.
- $i = 3, a = [1, 2, 5]$. If we perform the operation once on the 3rd element, $a = [1, 2, 4]$, which is a good array.
- $i = 4, a = [1, 2, 4]$. Since a is still good, we do not need to perform any operations.

In the fourth test case, a will remain good no matter what element we remove, so the answer for each index is 0.

H. Treasure Map Destruction (Counting Version)

2 seconds, 256 megabytes

Bessie the Buccaneer has plundered the six or seven seas for decades, but now she needs to find a place to bury her treasure!

There are n islands in a line, and Bessie buried her treasure on some number of islands. It is guaranteed that Bessie buried treasure on at least one island.

In order to remember where she buried her treasure, she created a treasure map. The treasure map can be represented as an array of n integers, where a_i indicates the minimum number of islands that Bessie would need to travel through to get to an island with treasure if she started at island i , assuming she can only travel to adjacent islands.

However, after Bessie buried her treasure, Elsie the Evil Explorer tried to steal Bessie's treasure map! Thankfully, Elsie was unsuccessful, but in the chaos, parts of Bessie's treasure map got destroyed, meaning that some of the elements of a were set to -1 ! Panicked, Bessie gives you the destroyed array b , and wants to know how many possible distinct sets of islands she could have placed her treasures on. Two sets A, B are distinct if there exists at least one element in A that is not in B or vice versa.

It's possible that Bessie messed up while writing the treasure map, so there could be no possible set of islands on which she could have placed her treasure.

Input

The first line of the input contains t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains n ($1 \leq n \leq 2 \cdot 10^5$) — the number of islands.

The second line contains $b_1, b_2, \dots, b_n$ ($-1 \leq b_i \leq n - 1$) — the destroyed array of distances. If $b_i = -1$, the element was destroyed.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each testcase, output the number of possible distinct sets of islands that Bessie could have placed her treasure on mod $10^9 + 7$.

input
12 5 0 1 -1 -1 0 3 -1 0 2 5 -1 1 -1 1 -1 5 -1 -1 -1 -1 -1 5 -1 2 -1 3 -1 7 2 1 0 1 0 1 2 1 -1 3 1 -1 1 1 0 4 3 -1 -1 -1 6 -1 -1 0 -1 2 4 10 -1 1 -1 -1 1 1 -1 -1 2 -1
output
4 0 5 31 0 1 1 1 1 1 1 0 3

In the first test case, while we know that Bessie placed treasure at the first and fifth islands, Bessie could have also placed her treasure at the third island, fourth island, both the third and fourth islands, or neither the third nor fourth island, leading to 4 possible sets of island locations.

In the fourth test case, since the entire map was destroyed, all $2^5 - 1 = 31$ **non-empty** sets of islands are possible.

Hints: Codeforces Round 1119 (Div. 3)

Taken from the official editorial, not written by us. Read one hint at a time.

A. Moo Language School (editorial, first line)

Farmer John can choose to build on every field that Farmer Nhoj does not own.

B. Minus Two (official hints)

Hint 1: When comparing two numbers that are initially not equal, the only way they can become equal after some number of operations is if at least one of them has an operation performed on it when it is < 2 .

Hint 2: If two elements are equal at some point, they'll be equal no matter how many more operations are performed. Therefore, it suffices to look at the end behavior of each element.

C. 101 (official hints)

Hint 1: If a -1 is in between two 1 s, it would only hurt the answer to make it a 1 .

Hint 2: If a -1 is not in between two 1 s, the -1 would only be included in a subarray satisfying the conditions if it was an endpoint.

D. MEX Multiset (official hints)

Hint 1: If there is only one 0 in the array, then exactly one set will have a mex greater than zero

Hint 2: $MEX(A) = MEX(B)$, $MEX(C) = 0$ satisfies the constraints.

E. Treasure Map Destruction (Constructive Version) (official hints)

Hint 1: If an element x at index i is not equal to -1 , there must be a treasure at either island $i + x$ or $i - x$

Hint 2: If an element x at index i is not equal to -1 , there must no treasures in the exclusive range of indices $(i + x, i - x)$

F. Binary Bubble Sort Inversions (official hints)

Hint 1: Look at performing some bubbles and reverse bubbles on small arrays. How many elements are moved?

Hint 2: A bubble will result is the leftmost 1 being moved all the way to the right. Similarly, a reverse bubble will result in the rightmost 0 being moved all the way to the left.

Hint 3: Since prefix zeroes and suffix ones don't matter, if we continuously remove them, then a bubble removes the last element of the array and a reverse bubble removes the first element of the array. Is there a way to remove the first and last elements of an array in constant time?

G. Index Removal (official hints)

Hint 1: Ignoring indexes 1 and n , if we remove index i , if we perform an operation on index j , all elements between $[i, j]$ will also have an operation performed on them.

Hint 2: If we remove index i , and we have to reduce the element at index $i + 1$, $i + 2$, etc, what will we subtract it to?

H. Treasure Map Destruction (Counting Version) (official hints)

Hint 1: Let's consider all locations that we have not restricted: we can separate them into 3 categories: the position is forced to be a treasure, the position has no undestroyed that depend on it, and all other positions.

Hint 2: The islands that are considered "non-restricted" form a chain of locations, with each non-restricted chain having a treasure between them. What must be true of the locations in this chain such that no condition is violated?

Hint 3: For every pair of adjacent locations, there must be at least one treasure at one of those locations.

Source: <https://codeforces.com/blog/entry/156457>